

Transformada discreta de Fourier

Tema 3 · Parte 4: FFT y convolución rápida

Jesús Chacón Sombría

Departamento de Arquitectura de Computadores y Automática
Universidad Complutense de Madrid

Grado en Ingeniería Electrónica de Comunicaciones
Curso 2026-2027

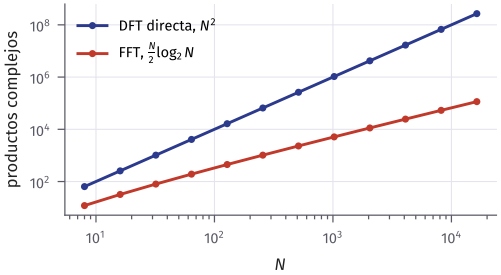
Contenido



1. La FFT
2. Convolución rápida

¿Por qué hace falta un algoritmo rápido?

- La DFT directa, $X[k] = \sum_n x[n] W_N^{kn}$, necesita N^2 productos y $N(N-1)$ sumas complejas.
- Con $N = 1024$: un millón de productos por espectro; con audio a 48 kHz y un espectro cada 10 ms, cien millones por segundo.
- La FFT (*fast Fourier transform*) calcula exactamente lo mismo con unos $\frac{N}{2}\log_2 N$ productos: 5120 para $N = 1024$, 200 veces menos.
- Cooley y Tukey (1965), aunque Gauss usó la misma idea hacia 1805.



N	N^2	$\frac{N}{2}\log_2 N$	ganancia
64	4096	192	21
1024	$1{,}05 \cdot 10^6$	5120	205
2^{16}	$4{,}3 \cdot 10^9$	$5{,}2 \cdot 10^5$	8192

Diezmado en el tiempo (base 2)

Sea $N = 2^p$. Separamos las muestras pares ($n = 2r$) y las impares ($n = 2r+1$), y usamos $W_N^2 = W_{N/2}$:

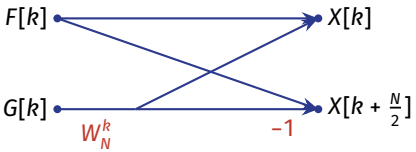
$$X[k] = \sum_{r=0}^{N/2-1} x[2r]\, W_N^{2rk} + \sum_{r=0}^{N/2-1} x[2r+1]\, W_N^{(2r+1)k} = \underbrace{\sum_{r=0}^{N/2-1} x[2r]\, W_{N/2}^{rk}}_{F[k]} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1]\, W_{N/2}^{rk}}_{G[k]}$$

- $F[k]$ y $G[k]$ son las DFT de $N/2$ puntos de las muestras pares e impares; son periódicas de periodo $N/2$.
- Como $W_N^{k+N/2} = -W_N^k$, basta calcularlas para $k = 0, \dots, N/2-1$:

Mariposa

$$X[k] = F[k] + W_N^k\, G[k], \qquad X[k+N/2] = F[k] - W_N^k\, G[k], \qquad k = 0, \dots, \frac{N}{2} - 1$$

La mariposa y la recursión



Un producto complejo y dos sumas por cada par de salidas.

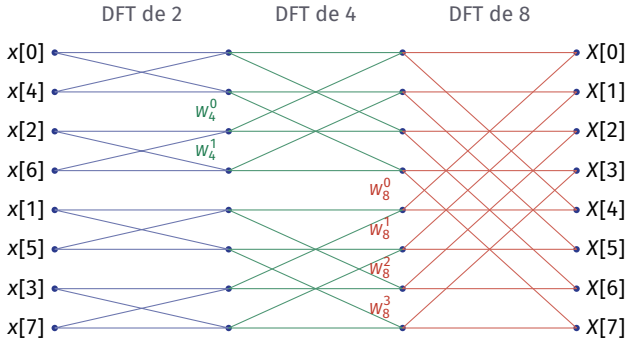
- Las DFT de $N/2$ puntos se dividen igual, hasta llegar a DFT de 1 punto ($X[0] = x[0]$).
- $\log_2 N$ etapas de $N/2$ mariposas:

$$\frac{N}{2}\log_2 N \text{ productos}, \qquad N\log_2 N \text{ sumas}.$$

- La entrada queda en orden de bits invertidos: con $N = 8$, el índice $1 = 001_2$ va a la posición $100_2 = 4$.

n	0	1	2	3	4	5	6	7
binario	000	001	010	011	100	101	110	111
invertido	000	100	010	110	001	101	011	111
posición	0	4	2	6	1	5	3	7

FFT de 8 puntos



Tres etapas de cuatro mariposas. En cada mariposa, la rama inferior se multiplica por W_m^k y la salida inferior lleva un signo menos.

▶ Herramienta interactiva: `T3-fft-mariposa`

Variantes y la FFT en la práctica

- Diezmado en frecuencia: se separan $X[k]$ pares e impares; la salida queda en bits invertidos.
- Otras bases (4, 8, mixtas) y algoritmos para cualquier N (Bluestein, Rader). Las bibliotecas (FFTW, la de NumPy y la de Matlab) eligen el método según N; con N potencia de 2 o con factores pequeños es más rápida.
- La IFFT es una FFT con el signo cambiado:

$$x[n] = \frac{1}{N}\Big(\sum_k X^*[k] W_N^{kn}\Big)^* = \frac{1}{N}\left(\mathrm{FFT}\{X^*\}\right)^*$$

- Con señales reales se aprovecha la simetría (`rfft`): la mitad de trabajo.
- Se puede calcular *en el sitio* (sin memoria adicional) y en coma fija; es la operación estrella de los DSP.

En código: FFT en base 2

Python

```python
import numpy as np

rng = np.random.default_rng(0)
x = rng.standard_normal(1024)
N = len(x)
b = int(np.log2(N))              # N = 2^b
# Reordenación por inversión de bits
r = [int(f"{i:0{b}b}"[::-1], 2)
     for i in range(N)]
X = x[r].astype(complex)
m = 2                            # tamaño DFT
while m <= N:                    # b etapas
    B = X.reshape(N // m, m)     # bloques
    F, G = B[:, :m//2], B[:, m//2:]
    W = np.exp(-2j*np.pi*np.arange(m//2)/m)
    X = np.hstack([F + W*G, F - W*G]).ravel()
    m *= 2                       # mariposas
print(np.allclose(X, np.fft.fft(x)))
```

Matlab

```matlab
x = randn(1024, 1);
N = length(x);
% Reordenación por inversión de bits
X = x(bitrevorder(1:N));
m = 2;                           % tamaño DFT
while m <= N                  % log2(N) etapas
    B = reshape(X, m, N/m);      % bloques
    F = B(1:m/2, :);
    G = B(m/2+1:end, :);
    W = exp(-2j*pi*(0:m/2-1).'/m);
    B = [F + W.*G; F - W.*G];    % mariposas
    X = B(:);
    m = 2*m;
end
disp(max(abs(X - fft(x))))       % ≈ 1e-12
```

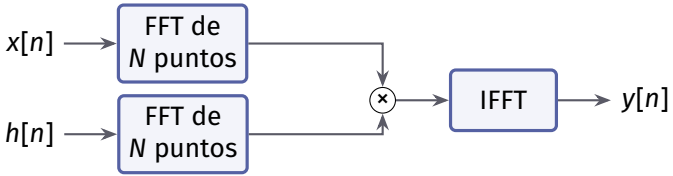
Cada etapa combina bloques de $m/2$ en bloques de m con mariposas, tras reordenar la entrada por inversión de bits.

Contenido

1 La FFT

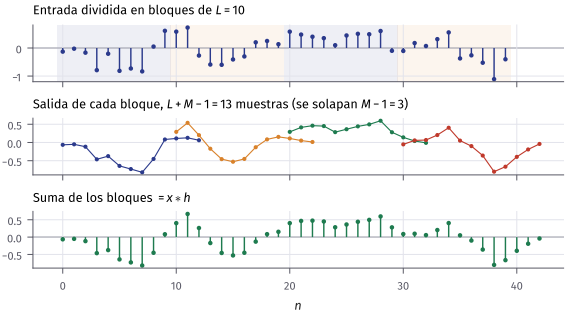
2 Convolución rápida

Convolución con la FFT



- Con $N \geq L + M - 1$ (rellenando con ceros) se obtiene la convolución lineal (parte 2).
- Coste: tres FFT y N productos, unos $\frac{3}{2} N \log_2 N + N$, frente a LM de la suma directa.
- Con $L = M = 1000$: 10^6 productos directos frente a unos $3{,}6 \cdot 10^4$ con $N = 2048$.
- Para filtros cortos ($M \lesssim 50$) la suma directa sigue siendo competitiva.
- $H[k]$ se calcula una sola vez si se filtran muchas señales con el mismo h.

Señales largas: solapamiento y suma



Si $x[n]$ es muy larga (o no termina), se divide en bloques de L muestras, $x[n] = \sum_i x_i[n - iL]$:

- Cada bloque se convoluciona con h por FFT de $N \geq L + M - 1$ puntos.
- Las salidas, de $L + M - 1$ muestras, se solapan $M - 1$ y se suman:

$$y[n] = \sum_i y_i[n - iL], \quad y_i = x_i * h$$

- Retardo de un bloque: apto para tiempo real.

Señales largas: solapamiento y almacenamiento

- Bloques de entrada de N muestras que se solapan $M-1$ con el anterior.
- Se calcula la convolución circular de N puntos de cada bloque con h.
- Las $M-1$ primeras muestras están contaminadas por el plegado circular: se descartan. Las $N-M+1$ restantes coinciden con la convolución lineal y se guardan.

	Solapamiento y suma	Solapamiento y almacenamiento
Bloques de entrada	L muestras, sin solape, + ceros	N muestras, solape $M-1$
Convolución	lineal (por relleno)	circular
Salida	se suman los solapes	se descartan $M-1$ muestras

Las dos dan el mismo resultado con el mismo coste. `scipy.signal.oaconvolve` *y* `fftfilt` *de Matlab las implementan.*

▶ Herramienta interactiva: `T3-solapamiento-suma`

En código: solapamiento y suma

Python

```python
import numpy as np

rng = np.random.default_rng(1)
x = rng.standard_normal(10_000)
h = np.ones(31) / 31            # FIR, M = 31
L = 226                          # bloque
M = len(h)
N = L + M - 1                    # 256 = 2^8
H = np.fft.fft(h, N)
y = np.zeros(len(x) + M - 1)
for i in range(0, len(x), L):
    xb = x[i:i+L]
    Yb = np.fft.fft(xb, N) * H
    yb = np.real(np.fft.ifft(Yb))
    n = min(N, len(y) - i)
    y[i:i+n] += yb[:n]      # solapa y suma
print(np.allclose(y, np.convolve(x, h)))
```

Matlab

```matlab
x = randn(1, 10000);
h = ones(1, 31) / 31;           % FIR, M = 31
L = 226;                         % bloque
M = length(h);
N = L + M - 1;                   % 256 = 2^8
H = fft(h, N);
y = zeros(1, length(x) + M - 1);
for i = 1:L:length(x)
    xb = x(i:min(i+L-1, end));
    yb = real(ifft(fft(xb, N) .* H));
    n = min(N, length(y) - i + 1);
    y(i:i+n-1) = y(i:i+n-1) + yb(1:n);
end
disp(max(abs(y - conv(x, h))))  % ≈ 1e-15
```

Bloques de 226 muestras y FFT de $N = 256$: el resultado coincide con la convolución directa.

¿Y si el sistema es IIR?

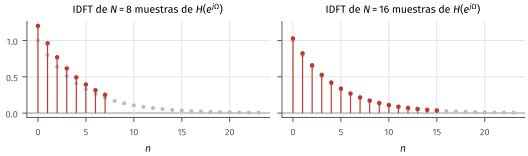
Si $h[n]$ es infinita, la IDFT de N muestras de $H(e^{j\Omega})$ es su extensión periódica:

$$h_N[n] = \sum_r h[n - rN]$$

$h[n] = a^n u[n]$, $|a| < 1$

$$H[k] = \frac{1}{1 - aW_N^k} \text{ y } h_N[n] = \frac{a^n}{1 - a^N},\ 0 \le n < N.$$

- Con N grande ($a^N \approx 0$) el error es despreciable: la DFT filtra aproximadamente con un IIR.
- Lo exacto (y más barato) es la ecuación en diferencias (Tema 4).



$a = 0{,}8$: *con* $N = 8$, $h_N[0] = 1/(1 - 0{,}8^8) = 1{,}20$; *con* $N = 16$, $1{,}03$.

Conceptos importantes

- FFT: misma DFT en $\frac{N}{2}\log_2 N$ productos.
- Diezmado en el tiempo:
  $X[k] = F[k] + W_N^k G[k]$,
  $X[k + \frac{N}{2}] = F[k] - W_N^k G[k]$.
- $\log_2 N$ etapas de mariposas; entrada en bits invertidos.
- IFFT con la misma FFT.
- Convolución rápida: FFT, producto, IFFT con $N \geq L + M - 1$.
- Señales largas: solapamiento y suma o solapamiento y almacenamiento.
- Con IIR, la DFT da h periodizada: aproximación válida si N es grande.

Apuntes del Tema 3, sección 5 · Hoja de ejercicios, ejercicios 14–16.